

microPAM Python Tools — User Guide

This document describes the three Python companion tools supplied with microPAM V4:

Tool	File	Purpose
micropam_control	Python/micropam_control.py	Configure and control a connected PAM over USB serial
micropam_browser	Python/micropam_browser.py	Browse, inspect, and visualise recorded data files
micropam_reader	Python/micropam_reader.py	Python API / command-line decoder for all file formats

Requirements

Python 3.10 or newer. A ready-to-use virtual environment is provided at Python/.venv.

Install or update dependencies:

```
Python\.venv\Scripts\pip.exe install -r Python\requirements.txt
```

Package	Minimum version	Used by
numpy	2.0	reader, browser
matplotlib	3.9	browser (data plots)
pyserial	3.5	control
scipy	1.13	optional, not required at runtime

To use the tools from PyCharm, set the project interpreter to Python\.venv\Scripts\python.exe.

micropam_control

Overview

micropam_control.py is a tkinter GUI that communicates with a microPAM device over its USB serial port. It implements the ? (query) / ! (set) serial protocol defined in src/menu.cxx, presenting all device parameters as labelled fields that can be read and written with a single click.

Running

```
Python\.venv\Scripts\python.exe Python\micropam_control.py
```

Connection

1. Connect the PAM to the PC via USB.
2. Open the **Port** drop-down and select the device COM port (e.g. COM5).
3. Click **Connect**. The UID field is populated automatically.

The GUI connects at 115200 baud, but microPAM uses USB CDC so the baud rate is ignored by the device.

Click **Disconnect** before unplugging or when finished.

Parameter panels

Recording timing

Field	Command	Description
t_acq	?a / !a	Acquisition window length in seconds
t_on	?o / !o	On-time per duty cycle in minutes
t_rep	?r / !r	Repeat interval; 0 = continuous

Signal

Field	Command	Description
fsamp	?f / !f	Sample rate (Hz)
again	?g / !g	ADC analogue gain (dB)

File metadata (written into WAV INFO chunks)

Field	Command	Description
Artist	?n / !n	Creator / organisation
Product	?p / !p	Project or instrument type
Name	?m / !m	Location or station ID
Start Time	?x / !x	Scheduled start time string

Hour windows h_rec[0]-h_rec[3] define up to four daily recording windows (start-stop hour pairs). Read with ?1...?4, write with !1...!4.

RTC / datetime

Button	Command	Action
getRTC	?d	Read MCU RTC time into the Datetime field
Set RTC	!d	Write the Datetime field to the MCU RTC
syncRTC	!d	Stamp current PC time into the field and send to MCU in one click

Bulk operations

Button	Action
Read all	Queries every ? command and fills the entire GUI
Start	Sends s — begins recording
Stop	Sends e — ends recording
Print params	Sends p — prints all parameters to the serial monitor

Button	Action
Reboot	Sends <code>b</code> — reboots the MCU

Hibernate Enter `n` hours and `m` minutes in the hibernate fields then click **Hibernate** to send `x<n>` and `y<m>` and put the device into low-power sleep.

Raw command entry The **Raw send** box accepts any single character or command string and sends it verbatim on Return. Responses appear in the serial monitor below.

Serial monitor

The full-width pane at the bottom shows all bytes received from the device. Lines of the form `key = value` are automatically parsed and, where a matching GUI field exists, the field is updated.

micropam_browser

Overview

`micropam_browser.py` is a tkinter file browser that lists microPAM data files (`.wav`, `.bin`, `.spc`, `.int`, `.vae`), shows key header parameters in a sortable table, and opens interactive matplotlib plots when a file is selected.

Running

Python\.`venv`\Scripts\python.exe Python\micropam_browser.py [folder]

If folder is omitted the browser opens in the current working directory.

File list

The table shows one row per file with the following columns:

Column	Content
File	Filename
Size	File size (bytes / KB / MB)
Modified	Last-modified timestamp
Mode	PROC_MODE number and label
fsamp	Sample rate (Hz)
nch	Number of channels
Dur / Size	Duration for mode 0 (computed from file size); raw data size for compressed modes

Click a column header to sort; click again to reverse.

Navigation

Control	Action
Up	Navigate to parent folder
Refresh	Re-scan the current folder
Open folder...	Pick a folder with a file-chooser dialog
Directory entry	Shows the current path (read-only)

Right-click menu

Right-click any row (or double-click, or press Return / v) to open:

Menu item	Action
Show header info	Opens a scrollable popup with all WAV and IKEY parameters parsed from the file header
View data	Opens a matplotlib plot window for the file (see below)
Open containing folder	Opens the file's parent folder in Windows Explorer

Header popup

Displays all parameters extracted from the 512-byte WAV header:

- File path, size, and modification time
- WAV format: channels, sample rate, bit depth
- IKEY acquisition parameters: PROC_MODE, UID, version, sample rate, gain, SHIFT, NBUF_PROC, NAVG, t_acq, t_on, t_rep, h_rec
- Recording start time parsed from the filename
- INFO chunk tags

Scroll horizontally and vertically; press **Escape** or **Close** to dismiss.

Data viewer

The **View data** window embeds a matplotlib figure with a standard navigation toolbar (zoom, pan, home, save). Layout depends on PROC_MODE:

Mode 0 — Raw WAV / Mode 1 — Compressed PCM

- **Top half** (stacked): one time-series subplot per channel, all sharing the same x-axis (time in seconds).
- **Bottom half**: STFT spectrogram of channel 0 (512-point Hann window, 50 % overlap, 60 dB dynamic range), sharing the same x-axis as the time series above. A colour bar shows the dB scale.

Panning or zooming in time synchronises all panels.

Mode 2 — Compressed spectrum (SPC) nch spectrogram panels stacked vertically, one per channel. Each panel shows frequency (Hz) on the y-axis and frame index on the x-axis, colour-mapped to signal magnitude in dB. All panels share x and y axes; each has its own colour bar.

Mode 3 — Directional intensity (INT) Four panels stacked vertically, all sharing axes:

1. **|I|** — instantaneous intensity magnitude, $\log_{10}(|I| + 1)$, *inferno* colourmap.
2. **I_x** — x-component, signed log, *RdBu_r* colourmap (blue = negative, red = positive).
3. **I_y** — y-component, same scale.
4. **I_z** — z-component, same scale.

Each panel has a colour bar. The logarithmic scale is necessary because acoustic intensity data is highly sparse (~98 % of bins are near zero at any given moment).

Mode 4 — VAE classifier (VAE) Five time-series panels stacked vertically, all sharing the x-axis (time in seconds from the start of the file):

1. **Detection excess** — $D_{snr} - DETECT_THR$. Positive values (shaded) indicate frames where the background-normalised intensity exceeded the detection threshold (default 3.0). A dashed red line marks zero.
- 2-5. **VAE 0-3** — per-VAE anomaly score $\|\mu - \mu_{bg}\| \times (err / err_{bg})$. All four panels share the same y-axis, scaled to 110 % of the overall maximum score. A grey dashed line on each panel marks the noise-frame mean + 3σ threshold for that VAE, computed from frames where `signal_flag == 0`.

micropam_reader — Python API

`micropam_reader.py` is a self-contained library (no dependencies other than numpy) that reads all five microPAM file formats.

Quick start

```
from micropam_reader import MicroPAMFile

# Read header only
f = MicroPAMFile("recording.int")
print(f.info["proc_mode"], f.info["sample_rate"])

# Read all data at once
result = f.read_all()
data = result["data"]           # numpy array
millis = result["millis"]       # list of firmware timestamps (ms)

# Stream block by block (memory-efficient for large files)
with MicroPAMFile("recording.wav") as f:
    for block in f.iter_blocks():
        pcm = block["data"]     # int32 ndarray, shape (nframes, nch)
```

Return format by PROC_MODE

Mode	data dtype	data shape	Extra keys
0	int32	(nframes, nch)	—
1	int32	(nframes, nch)	—
2	int32	(nsamp×n_rec, nch)	—
3	int32	(nsamp×n_rec, nch)	—

Mode	data dtype	data shape	Extra keys
4	float32	alias for vae_mu	signal_flag, detection_excess, vae_detect, vae_mu, I

For mode 4, `read_all` returns a dict with:

Key	dtype	Shape	Content
millis	list[int]	(N,)	Firmware timestamp per frame (ms)
signal_flag	int8	(N,)	1 = detection, 0 = noise
detection_excess	float32	(N,)	Dsnr – DETECT_THR
vae_detect	float32	(N, 4)	Per-VAE anomaly score
vae_mu	float32	(N, 16)	VAE latent means (4 VAEs × 4 dims)
I	float32	(N, 3, 512)	Intensity matrix; zeros on noise frames

Command-line usage

Python\.[.venv](#)\Scripts\python.exe Python\micropam_reader.py <file> [block_words]

Prints header parameters and decoded data statistics (shape, min/max, record count).

microPAM V4 — Copyright © 2026 Walter Zimmer. Released under the MIT License.